zip Can Combine Lists

Walk several sequences in step, pair them, build dicts, unzip and transpose

Loop over two lists together

```
names  = ["Asha", "Ravi", "Meera"]
scores = [88, 72, 95]
for name, score in zip(names, scores):
    print(name, score)
```

OUTPUT

```
Asha 88
Ravi 72
Meera 95
```

Pairs by position

`zip` returns a lazy iterator of tuples: item 0 with item 0, item 1 with item 1...

```
list(zip("ab", [1, 2]))  # → [('a', 1), ('b', 2)]
```

Three or more sequences

```
cities = ["Pune", "Delhi", "Goa"]
for n, s, c in zip(names, scores, cities):
    print(f"{n} ({c}): {s}")
```

OUTPUT

```
Asha (Pune): 88
Ravi (Delhi): 72
Meera (Goa): 95
```

Different lengths

`zip` stops at the **shortest** input. `strict=True` (3.10+) raises `ValueError` on a mismatch; `zip_longest` pads instead.

```
list(zip([1, 2, 3], "ab"))  # → [(1, 'a'), (2, 'b')]
from itertools import zip_longest as zl
print(list(zl([1, 2, 3], "ab", fillvalue="-")))
```

OUTPUT

```
[(1, 'a'), (2, 'b'), (3, '-')]
```

Build a dictionary

```
lookup = dict(zip(names, scores))
print(lookup)
print(lookup["Ravi"])
```

OUTPUT

```
{'Asha': 88, 'Ravi': 72, 'Meera': 95}
72
```

Unzip

`zip(*pairs)` is the reverse: it splits a list of pairs back into columns.

```
pairs = [("a", 1), ("b", 2), ("c", 3)]
letters, nums = zip(*pairs)
print(letters, nums)
```

OUTPUT

```
('a', 'b', 'c') (1, 2, 3)
```

Compute across lists

```
q1, q2 = [10, 20, 30], [5, 15, 25]
[a + b for a, b in zip(q1, q2)]  # → [15, 35, 55]
sum(a * b for a, b in zip(q1, q2))  # → 1100
```

Compare neighbours

Zip a list with itself shifted by one.

```
temps = [20, 22, 21, 25]
diffs = [b - a for a, b in zip(temps, temps[1:])]
print(diffs)              # → [2, -1, 4]
```

Transpose rows to columns

```
matrix = [[1, 2, 3], [4, 5, 6]]
print(list(zip(*matrix)))
```

OUTPUT

```
[(1, 4), (2, 5), (3, 6)]
```

A zip object is single-use

```
z = zip("ab", [1, 2])
print(list(z))            # → [('a', 1), ('b', 2)]
print(list(z))            # → []
```

WATCH OUT Once consumed, a zip is empty. Store `list(zip(...))` if you need it twice.

REMEMBER

- `zip(a, b)` pairs items by **position** and returns a lazy iterator of tuples.
- It silently stops at the shortest input: check lengths or use `strict=True`.
- `dict(zip(keys, values))` is the idiomatic way to build a dict from two lists.
- `zip(*pairs)` unzips; `zip(*matrix)` transposes.
- Combine with `enumerate`: `for i, (a, b) in enumerate(zip(x, y))`.
- A zip can be consumed only once; wrap it in `list()` to reuse it.